

Advanced OOP: Placement Interview Notes

This file covers seven closely related OOP topics that frequently appear in placement interviews and low-level design discussions:

- Overloading
- Overriding
- Compile-time polymorphism
- Runtime polymorphism
- Association
- Aggregation
- Composition

The goal is not only to remember definitions, but to understand when each idea appears in real code and how to explain it clearly in interviews.

1. Polymorphism: The Big Picture

Polymorphism means "many forms".

In OOP, polymorphism allows the same operation, method name, or interface to behave differently depending on context.

There are two major types:

Type	Also Called	Decision Happens At	Main Mechanism
Compile-time polymorphism	Static polymorphism	Compile time	Method overloading, operator overloading
Runtime polymorphism	Dynamic polymorphism	Runtime	Method overriding, dynamic method dispatch

Simple mental model:

- Overloading: same method name, different parameter list.
 - Overriding: subclass gives a new implementation of a parent method.
 - Compile-time polymorphism: compiler decides which method to call.
 - Runtime polymorphism: object decides which method to call at runtime.
-

2. Method Overloading

Definition

Method overloading means defining multiple methods with the same name in the same class, but with different parameter lists.

The parameter list can differ by:

- Number of parameters
- Type of parameters
- Order of parameters

Return type alone is not enough to overload a method.

Example

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Usage:

```
Calculator calculator = new Calculator();  
  
calculator.add(2, 3);           // calls add(int, int)  
calculator.add(2, 3, 4);       // calls add(int, int, int)  
calculator.add(2.5, 3.5);      // calls add(double, double)
```

Why Overloading Is Useful

Overloading improves readability when several operations conceptually do the same thing but accept different inputs.

Examples:

```
print(String message)  
print(int number)  
print(double value)
```

```
createUser(String name)  
createUser(String name, String email)
```

```
createUser(String name, String email, String phone)
```

Rules Of Method Overloading

Valid overloading:

```
void show(int value) {}  
void show(String value) {}  
void show(int value, String label) {}  
void show(String label, int value) {}
```

Invalid overloading:

```
int getValue() {  
    return 10;  
}  
  
double getValue() {  
    return 10.5;  
}
```

This is invalid because only the return type differs. The compiler cannot decide which one to call from:

```
getValue();
```

Overloading And Type Promotion

Java may promote smaller numeric types when finding an overloaded method.

```
class Demo {  
    void show(int x) {  
        System.out.println("int");  
    }  
  
    void show(double x) {  
        System.out.println("double");  
    }  
}
```

```
Demo d = new Demo();  
d.show(10);    // int  
d.show(10.5); // double
```

If `show(int)` did not exist, `show(double)` could accept `10` after promotion.

Common promotion chain:

```
byte -> short -> int -> long -> float -> double  
char -> int
```

Overloading With Null

This is a common tricky interview question.

```
class Demo {  
    void show(String s) {  
        System.out.println("String");  
    }  
  
    void show(Object o) {  
        System.out.println("Object");  
    }  
}
```

```
Demo d = new Demo();  
d.show(null); // String
```

Why?

`null` can match both `String` and `Object`, but `String` is more specific.

But this causes ambiguity:

```
class Demo {  
    void show(String s) {}  
    void show(Integer i) {}  
}
```

```
d.show(null); // compile-time error: ambiguous
```

Because neither `String` nor `Integer` is more specific than the other.

Constructor Overloading

Constructors can also be overloaded.

```
class Student {
    String name;
    int age;

    Student() {
        this.name = "Unknown";
        this.age = 0;
    }

    Student(String name) {
        this.name = name;
        this.age = 0;
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Constructor overloading is useful when objects can be created with different levels of information.

Can Static Methods Be Overloaded?

Yes.

```
class Utility {
    static int max(int a, int b) {
        return a > b ? a : b;
    }

    static double max(double a, double b) {
        return a > b ? a : b;
    }
}
```

Static method overloading is resolved at compile time.

Is Overloading Compile-Time Polymorphism?

Yes.

The compiler selects the correct overloaded method based on the method signature.

Method signature includes:

- Method name
- Parameter types
- Parameter order
- Parameter count

Method signature does not include:

- Return type
- Access modifier
- `static`
- `final`
- `throws`

3. Operator Overloading

Definition

Operator overloading means giving special behavior to operators such as `+`, `-`, `*`, `/`, `==`, and so on.

Java does not support custom operator overloading, except built-in behavior like:

```
int a = 10 + 20;
String s = "Hello " + "World";
```

Here `+` works for both numeric addition and string concatenation.

C++ supports operator overloading:

```
class Complex {
public:
    int real, imag;

    Complex(int r, int i) {
        real = r;
        imag = i;
    }

    Complex operator+(const Complex& other) {
        return Complex(real + other.real, imag + other.imag);
    }
};
```

Interview Note

If asked "Does Java support operator overloading?", the best answer is:

Java does not allow programmers to define custom operator overloading. However, Java internally overloads `+` for numeric addition and string concatenation.

4. Method Overriding

Definition

Method overriding means a subclass provides its own implementation of a method already defined in its parent class.

The method name, return type, and parameters should match the parent method, with some allowed rules around covariant return types.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Usage:

```
Animal animal = new Dog();
animal.sound(); // Dog barks
```

Although the reference type is `Animal`, the actual object is `Dog`, so the `Dog` method runs.

Why Overriding Is Useful

Overriding allows specialized behavior in subclasses while preserving a common parent type.

Example:

```
abstract class Payment {
    abstract void pay(double amount);
}

class CreditCardPayment extends Payment {
    @Override
```

```
    void pay(double amount) {
        System.out.println("Paid using credit card: " + amount);
    }
}

class UpiPayment extends Payment {
    @Override
    void pay(double amount) {
        System.out.println("Paid using UPI: " + amount);
    }
}
```

```
Payment payment = new UpiPayment();
payment.pay(500);
```

The caller can use `Payment` without caring about the exact payment method.

Rules Of Method Overriding

For overriding:

- The method must be inherited from a parent class or interface.
- The method name must be the same.
- The parameter list must be the same.
- The return type must be the same or covariant.
- The access modifier cannot be more restrictive.
- The overriding method cannot throw broader checked exceptions.
- `private` methods cannot be overridden.
- `final` methods cannot be overridden.
- `static` methods are not overridden; they are hidden.
- Constructors cannot be overridden.

Access Modifier Rule

A subclass can make an inherited method more accessible, but not less accessible.

Valid:

```
class Parent {
    protected void show() {}
}

class Child extends Parent {
    @Override
    public void show() {}
}
```

Invalid:

```
class Parent {  
    public void show() {}  
}  
  
class Child extends Parent {  
    @Override  
    protected void show() {}  
}
```

The child cannot reduce visibility from `public` to `protected`.

Covariant Return Type

An overriding method can return a subclass of the parent method's return type.

```
class Animal {}  
class Dog extends Animal {}  
  
class Parent {  
    Animal getAnimal() {  
        return new Animal();  
    }  
}  
  
class Child extends Parent {  
    @Override  
    Dog getAnimal() {  
        return new Dog();  
    }  
}
```

This is valid because `Dog` is a subtype of `Animal`.

Private Methods And Overriding

Private methods are not inherited, so they cannot be overridden.

```
class Parent {  
    private void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    private void show() {  
        System.out.println("Child");  
    }  
}
```

```
}  
}
```

This is not overriding. These are two separate private methods.

Static Methods And Method Hiding

Static methods belong to the class, not to objects.

So static methods are hidden, not overridden.

```
class Parent {  
    static void show() {  
        System.out.println("Parent static");  
    }  
}  
  
class Child extends Parent {  
    static void show() {  
        System.out.println("Child static");  
    }  
}
```

```
Parent obj = new Child();  
obj.show(); // Parent static
```

The method is chosen based on the reference type, not the object type.

Final Methods

Final methods cannot be overridden.

```
class Parent {  
    final void show() {}  
}  
  
class Child extends Parent {  
    // void show() {} // compile-time error  
}
```

Use `final` when the parent class wants to prevent subclasses from changing a method's behavior.

5. Overloading Vs Overriding

Basis	Overloading	Overriding
Meaning	Same method name with different parameters	Subclass redefines parent method
Polymorphism type	Compile-time	Runtime
Class relationship required?	No	Yes, inheritance required
Parameters	Must be different	Must be same
Return type	Can be different, but not alone	Same or covariant
Access modifier	No major restriction	Cannot be more restrictive
Static methods	Can be overloaded	Hidden, not overridden
Private methods	Can be overloaded in same class	Cannot be overridden
Final methods	Can be overloaded	Cannot be overridden
Constructor	Can be overloaded	Cannot be overridden
Decision made by	Compiler	JVM/runtime

Quick Example

Overloading:

```
class Printer {  
    void print(String text) {}  
    void print(int number) {}  
}
```

Overriding:

```
class Parent {  
    void print() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void print() {  
        System.out.println("Child");  
    }  
}
```

6. Compile-Time Polymorphism

Definition

Compile-time polymorphism means the method to be executed is decided by the compiler before the program runs.

It is also called:

- Static polymorphism
- Early binding
- Static binding

Achieved By

Common mechanisms:

- Method overloading
- Constructor overloading
- Operator overloading in languages like C++

Example

```
class MathUtils {  
    int multiply(int a, int b) {  
        return a * b;  
    }  
  
    double multiply(double a, double b) {  
        return a * b;  
    }  
}
```

```
MathUtils utils = new MathUtils();  
utils.multiply(2, 3);           // compiler picks int version  
utils.multiply(2.5, 3.5);      // compiler picks double version
```

Why It Is Called Compile-Time

The compiler knows the argument types at compile time:

```
multiply(2, 3)
```

Both arguments are integers, so the compiler selects:

```
multiply(int, int)
```

Advantages

- Faster than runtime polymorphism because binding is early.
- Improves readability when operations are conceptually similar.
- Reduces need for different method names.
- Useful for API design.

Disadvantages

- Cannot choose behavior based on actual runtime object type.
- Too many overloads can make code confusing.
- Ambiguous overloads can cause compile-time errors.

Interview Line

Compile-time polymorphism allows the same method name to represent different operations, and the compiler decides which version to call based on the method signature.

7. Runtime Polymorphism

Definition

Runtime polymorphism means the method to be executed is decided at runtime based on the actual object.

It is also called:

- Dynamic polymorphism
- Late binding
- Dynamic binding
- Dynamic method dispatch

Achieved By

Runtime polymorphism is achieved using:

- Method overriding
- Inheritance
- Interfaces
- Abstract classes

Example With Inheritance

```
class Shape {  
    void draw() {  
        System.out.println("Drawing shape");  
    }  
}
```

```
    }  
}  
  
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing circle");  
    }  
}  
  
class Rectangle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing rectangle");  
    }  
}
```

```
Shape shape;  
  
shape = new Circle();  
shape.draw(); // Drawing circle  
  
shape = new Rectangle();  
shape.draw(); // Drawing rectangle
```

Same reference type:

Shape

Different actual object types:

Circle
Rectangle

Different behavior at runtime.

Example With Interface

```
interface NotificationSender {  
    void send(String message);  
}  
  
class EmailSender implements NotificationSender {  
    @Override  
    public void send(String message) {
```

```
        System.out.println("Email: " + message);
    }
}

class SmsSender implements NotificationSender {
    @Override
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}
```

```
class NotificationService {
    private final NotificationSender sender;

    NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    void notifyUser(String message) {
        sender.send(message);
    }
}
```

```
NotificationSender sender = new EmailSender();
NotificationService service = new NotificationService(sender);
service.notifyUser("Your order has shipped");
```

This design is flexible because `NotificationService` depends on an interface, not a concrete class.

Dynamic Method Dispatch

Dynamic method dispatch is the process where the JVM decides at runtime which overridden method to call.

```
Animal animal = new Dog();
animal.sound();
```

Reference type:

Animal

Object type:

```
Dog
```

Called method:

```
Dog.sound()
```

Why Runtime Polymorphism Is Important In LLD

Runtime polymorphism helps build extensible systems.

Example:

```
interface DiscountStrategy {
    double apply(double amount);
}

class NoDiscount implements DiscountStrategy {
    public double apply(double amount) {
        return amount;
    }
}

class FestivalDiscount implements DiscountStrategy {
    public double apply(double amount) {
        return amount * 0.80;
    }
}

class StudentDiscount implements DiscountStrategy {
    public double apply(double amount) {
        return amount * 0.90;
    }
}
```

```
class BillingService {
    private final DiscountStrategy discountStrategy;

    BillingService(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    double calculateFinalAmount(double amount) {
        return discountStrategy.apply(amount);
    }
}
```

This avoids writing:

```
if (discountType.equals("FESTIVAL")) {  
    ...  
} else if (discountType.equals("STUDENT")) {  
    ...  
}
```

Instead, each discount behavior is placed in its own class.

This is related to the Strategy Design Pattern.

Advantages

- Makes code flexible and extensible.
- Supports interface-based design.
- Helps follow Open/Closed Principle.
- Reduces large conditional blocks.
- Enables design patterns like Strategy, Factory, Template Method, and State.

Disadvantages

- Slight runtime overhead due to dynamic dispatch.
- Can be harder to trace than direct method calls.
- Poorly designed inheritance hierarchies can become complex.

Interview Line

Runtime polymorphism allows a parent reference to call a child class's overridden method, with the actual method resolved at runtime based on the object type.

8. Compile-Time Vs Runtime Polymorphism

Basis	Compile-Time Polymorphism	Runtime Polymorphism
Also called	Static polymorphism	Dynamic polymorphism
Binding	Early binding	Late binding
Resolved by	Compiler	JVM/runtime
Achieved using	Overloading	Overriding
Inheritance required?	No	Yes
Method parameters	Usually different	Same
Flexibility	Less flexible	More flexible

Basis	Compile-Time Polymorphism	Runtime Polymorphism
Speed	Usually faster	Slightly slower
Example	<code>add(int, int)</code> and <code>add(double, double)</code>	<code>Animal animal = new Dog();</code> <code>animal.sound();</code>

Memory Hook

Overloading = compile time = same name, different arguments.
Overriding = runtime = same method, different class behavior.

9. Association

Definition

Association is a general relationship between two classes where one class uses, knows about, or communicates with another class.

It represents a "has a relationship with" or "uses a" connection.

Association is the broadest relationship among association, aggregation, and composition.

Example

```
class Teacher {
    void teach(Student student) {
        System.out.println("Teaching " + student.getName());
    }
}

class Student {
    private String name;

    Student(String name) {
        this.name = name;
    }

    String getName() {
        return name;
    }
}
```

Here, `Teacher` is associated with `Student` because the teacher uses a student object.

Types Of Association

Association can be:

- One-to-one
- One-to-many
- Many-to-one
- Many-to-many

One-To-One Association

Example:

Person -> Passport

One person has one passport.

```
class Person {  
    private Passport passport;  
}  
  
class Passport {  
    private String passportNumber;  
}
```

One-To-Many Association

Example:

Department -> Employees

One department has many employees.

```
class Department {  
    private List<Employee> employees;  
}  
  
class Employee {  
    private String name;  
}
```

Many-To-One Association

Example:

Many employees -> One department

```
class Employee {  
    private Department department;  
}
```

Many-To-Many Association

Example:

Students <-> Courses

One student can enroll in many courses, and one course can have many students.

```
class Student {  
    private List<Course> courses;  
}  
  
class Course {  
    private List<Student> students;  
}
```

Association In UML

Association is usually shown with a plain line:

Teacher ----- Student

With multiplicity:

Teacher 1 ----- * Student

This means one teacher is associated with many students.

Interview Line

Association is a relationship where one class is connected to another class, but both objects can usually exist independently.

10. Aggregation

Definition

Aggregation is a specialized form of association that represents a weak "has-a" relationship.

In aggregation:

- One object contains or refers to another object.
- The contained object can exist independently.
- The lifecycle of the child object does not depend on the parent object.

Example

Department has Professors.

If the department is removed, the professors can still exist.

```
class Professor {
    private String name;

    Professor(String name) {
        this.name = name;
    }
}

class Department {
    private String name;
    private List<Professor> professors;

    Department(String name, List<Professor> professors) {
        this.name = name;
        this.professors = professors;
    }
}
```

Usage:

```
Professor p1 = new Professor("Asha");
Professor p2 = new Professor("Raman");

List<Professor> professors = List.of(p1, p2);

Department cse = new Department("CSE", professors);
```

The `Professor` objects are created outside the `Department` and passed into it. This suggests they can exist independently.

Real-World Examples

Aggregation examples:

- Department has professors.
- Team has players.
- Library has books.
- Company has employees.
- Playlist has songs.

In all these cases, the contained object may continue to exist if the container is deleted.

Aggregation In UML

Aggregation is shown using a hollow diamond.

```
Department <----- Professor
```

The hollow diamond is placed near the whole or container side.

Key Idea

Aggregation means:

```
Whole has parts, but parts can live without the whole.
```

Interview Line

Aggregation is a weak has-a relationship where the child object can exist independently of the parent object.

11. Composition

Definition

Composition is a stronger form of association that represents a strong "has-a" relationship.

In composition:

- One object owns another object.
- The child object is usually created inside the parent.
- The child object's lifecycle depends on the parent object.
- If the parent is destroyed, the child usually cannot exist meaningfully.

Example

```
House has Rooms.
```

If the house is destroyed, its rooms do not independently exist as rooms of that house.

```
class Room {
    private String type;

    Room(String type) {
        this.type = type;
    }
}

class House {
    private List<Room> rooms;

    House() {
        rooms = new ArrayList<>();
        rooms.add(new Room("Bedroom"));
        rooms.add(new Room("Kitchen"));
    }
}
```

Here, `House` creates and owns its `Room` objects.

Another Example

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private final Engine engine;

    Car() {
        this.engine = new Engine();
    }

    void startCar() {
        engine.start();
    }
}
```

This can be treated as composition if the `Engine` is owned exclusively by the `Car`.

However, in real-world modeling, engine replacement may complicate this. In interviews, focus on lifecycle dependency.

Real-World Examples

Composition examples:

- House has rooms.
- Human body has heart.
- Order has order items.
- Folder has files in a file-system model.
- Computer has motherboard.

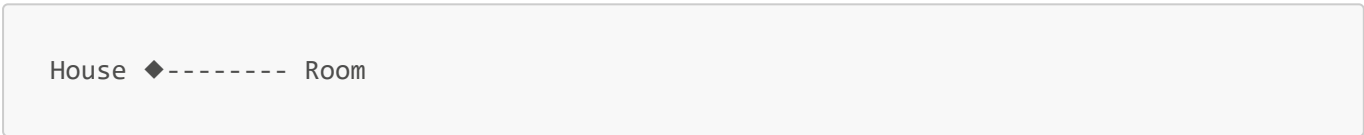
The important question:

Can the part exist independently in the domain model?

If no, it is probably composition.

Composition In UML

Composition is shown using a filled diamond.



The filled diamond is placed near the whole or owner side.

Key Idea

Composition means:

Whole owns parts, and parts depend on the whole.

Interview Line

Composition is a strong has-a relationship where the child object's lifecycle depends on the parent object.

12. Association Vs Aggregation Vs Composition

Basis	Association	Aggregation	Composition
Meaning	General relationship	Weak has-a relationship	Strong has-a relationship
Strength	Weak/general	Medium	Strong
Ownership	No clear ownership	Parent has child, but weak ownership	Parent strongly owns child
Lifecycle dependency	Usually independent	Child can exist independently	Child depends on parent
UML symbol	Plain line	Hollow diamond	Filled diamond

Basis	Association	Aggregation	Composition
Example	Teacher teaches Student	Department has Professors	House has Rooms
Object creation	Anywhere	Usually outside parent	Often inside parent
Deleting parent	Child usually survives	Child survives	Child usually does not survive meaningfully

Simple Memory Trick

Association: A uses B.
 Aggregation: A has B, but B can live alone.
 Composition: A owns B, and B depends on A.

Placement Interview Example

Suppose you are designing a university system.

Association:

Teacher teaches Student.

A teacher and student are related, but neither owns the other.

Aggregation:

Department has Professors.

A professor can exist even if the department changes or is removed from the system.

Composition:

University has Departments.

In some domain models, departments may not exist independently outside a university. If that is the assumption, it is composition.

Important: the same real-world example can be modeled differently depending on the system requirements. In interviews, explain your assumption.

13. Inheritance Vs Composition

This is another important placement topic.

Inheritance

Inheritance represents an "is-a" relationship.

```
class Dog extends Animal {}
```

A dog is an animal.

Composition

Composition represents a "has-a" relationship.

```
class Car {  
    private Engine engine;  
}
```

A car has an engine.

When To Use Inheritance

Use inheritance when:

- There is a true is-a relationship.
- The child class can fully substitute the parent class.
- You need runtime polymorphism.
- The parent class defines common behavior.

Example:

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    private double radius;  
  
    @Override  
    double area() {  
        return Math.PI * radius * radius;  
    }  
}
```

When To Use Composition

Use composition when:

- One class needs behavior from another class.
- You want flexibility.
- You want to avoid deep inheritance chains.
- You want to change behavior by replacing components.

Example:

```
interface PaymentMethod {  
    void pay(double amount);  
}  
  
class CheckoutService {  
    private final PaymentMethod paymentMethod;  
  
    CheckoutService(PaymentMethod paymentMethod) {  
        this.paymentMethod = paymentMethod;  
    }  
  
    void checkout(double amount) {  
        paymentMethod.pay(amount);  
    }  
}
```

This design uses composition plus runtime polymorphism.

Prefer Composition Over Inheritance

This is a common design principle.

It does not mean inheritance is bad.

It means:

- Do not use inheritance only to reuse code.
- Use inheritance when the relationship is truly is-a.
- Use composition when you want flexible behavior.

Bad inheritance example:

```
class Stack extends ArrayList<Integer> {  
}
```

This is problematic because a stack should expose only stack operations like `push`, `pop`, and `peek`, but `ArrayList` exposes operations that can break stack behavior.

Better:

```
class Stack {  
    private List<Integer> items = new ArrayList<>();  
  
    void push(int item) {  
        items.add(item);  
    }  
  
    int pop() {  
        return items.remove(items.size() - 1);  
    }  
}
```

The stack has a list internally, but does not expose the full list API.

14. Interview Traps And Good Answers

Can We Overload The Main Method In Java?

Yes.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Standard main");  
        main(10);  
    }  
  
    public static void main(int x) {  
        System.out.println("Overloaded main: " + x);  
    }  
}
```

But the JVM starts execution only from:

```
public static void main(String[] args)
```

Can We Override The Main Method?

No meaningful runtime overriding happens for `main` because it is static.

Static methods are hidden, not overridden.

Can We Override A Constructor?

No.

Constructors are not inherited, so they cannot be overridden.

But constructors can be overloaded.

Can We Override A Private Method?

No.

Private methods are not inherited.

Can We Override A Final Method?

No.

Final methods cannot be overridden.

Can We Overload A Final Method?

Yes.

```
class Demo {  
    final void show() {}  
    final void show(int x) {}  
}
```

Can We Override A Static Method?

No.

Static methods are hidden, not overridden.

Can We Overload A Static Method?

Yes.

```
class Demo {  
    static void show() {}  
    static void show(int x) {}  
}
```

Does Overriding Depend On Reference Type Or Object Type?

Overriding depends on the object type at runtime.

```
Animal animal = new Dog();  
animal.sound(); // Dog's method
```

Does Overloading Depend On Reference Type Or Object Type?

Overloading depends on the compile-time reference type and argument types.

Example:

```
class Animal {}
class Dog extends Animal {}

class Demo {
    void show(Animal a) {
        System.out.println("Animal");
    }

    void show(Dog d) {
        System.out.println("Dog");
    }
}
```

```
Animal animal = new Dog();
Demo demo = new Demo();
demo.show(animal); // Animal
```

Even though the object is `Dog`, the reference type is `Animal`, so the overloaded method `show(Animal)` is chosen.

This is a very important interview trap.

15. Combined Example: Overloading And Overriding Together

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }

    void eat(String food) {
        System.out.println("Animal eats " + food);
    }
}

class Dog extends Animal {
    @Override
    void eat() {
        System.out.println("Dog eats");
    }

    void eat(int quantity) {
        System.out.println("Dog eats quantity: " + quantity);
    }
}
```

```
}  
}
```

```
Animal animal = new Dog();  
  
animal.eat();           // Dog eats  
animal.eat("meat");     // Animal eats meat  
// animal.eat(2);       // compile-time error
```

Why?

`animal.eat()` calls the overridden method, so runtime dispatch selects `Dog.eat()`.

`animal.eat("meat")` is available in `Animal`, and `Dog` did not override it.

`animal.eat(2)` fails because the reference type is `Animal`, and `Animal` does not have `eat(int)`.

If you write:

```
Dog dog = new Dog();  
dog.eat(2); // works
```

Then the compiler can see `Dog.eat(int)`.

16. LLD Usage Examples

Strategy Pattern Uses Runtime Polymorphism

```
interface SortingStrategy {  
    void sort(List<Integer> numbers);  
}  
  
class QuickSort implements SortingStrategy {  
    public void sort(List<Integer> numbers) {  
        System.out.println("Quick sort");  
    }  
}  
  
class MergeSort implements SortingStrategy {  
    public void sort(List<Integer> numbers) {  
        System.out.println("Merge sort");  
    }  
}  
  
class Sorter {  
    private SortingStrategy strategy;
```

```
Sorter(SortingStrategy strategy) {  
    this.strategy = strategy;  
}  
  
void sort(List<Integer> numbers) {  
    strategy.sort(numbers);  
}  
}
```

The `Sorter` does not care which algorithm is used. That decision is delegated to the strategy object.

Factory Pattern Often Returns Parent Type

```
interface Vehicle {  
    void drive();  
}  
  
class Car implements Vehicle {  
    public void drive() {  
        System.out.println("Driving car");  
    }  
}  
  
class Bike implements Vehicle {  
    public void drive() {  
        System.out.println("Driving bike");  
    }  
}  
  
class VehicleFactory {  
    static Vehicle createVehicle(String type) {  
        if (type.equals("CAR")) {  
            return new Car();  
        }  
        if (type.equals("BIKE")) {  
            return new Bike();  
        }  
        throw new IllegalArgumentException("Unknown vehicle type");  
    }  
}
```

```
Vehicle vehicle = VehicleFactory.createVehicle("CAR");  
vehicle.drive(); // runtime polymorphism
```

Composition In Service Design

```
class OrderService {  
    private final PaymentService paymentService;  
    private final InventoryService inventoryService;  
    private final NotificationService notificationService;  
  
    OrderService(  
        PaymentService paymentService,  
        InventoryService inventoryService,  
        NotificationService notificationService  
    ) {  
        this.paymentService = paymentService;  
        this.inventoryService = inventoryService;  
        this.notificationService = notificationService;  
    }  
}
```

`OrderService` has other services. This is composition at the design level.

17. How To Identify The Relationship In Interviews

Ask these questions:

Is It Is-A?

If yes, inheritance may be suitable.

```
Dog is an Animal.  
Car is a Vehicle.  
Circle is a Shape.
```

Is It Has-A?

If yes, association, aggregation, or composition may be suitable.

```
Car has Engine.  
House has Rooms.  
Department has Professors.
```

Does One Object Merely Use Another?

Likely association.

```
Doctor treats Patient.  
Teacher teaches Student.
```

```
Customer places Order.
```

Can The Child Exist Without The Parent?

If yes, likely aggregation.

```
Department has Professors.  
Team has Players.  
Library has Books.
```

Is The Child's Lifecycle Dependent On The Parent?

If yes, likely composition.

```
House has Rooms.  
Order has OrderItems.  
Body has Heart.
```

18. Common Mistakes

Mistake 1: Saying Overloading Depends On Return Type

Wrong:

```
Methods can be overloaded by changing only return type.
```

Correct:

```
Methods cannot be overloaded by return type alone.
```

Mistake 2: Saying Static Methods Are Overridden

Wrong:

```
Static methods can be overridden.
```

Correct:

Static methods are hidden, not overridden.

Mistake 3: Confusing Aggregation And Composition

Wrong:

Both simply mean has-a.

Correct:

Aggregation is weak has-a. Composition is strong has-a with lifecycle dependency.

Mistake 4: Using Inheritance For Code Reuse Only

Wrong:

```
class UserRepository extends DatabaseConnection {}
```

Better:

```
class UserRepository {  
    private DatabaseConnection connection;  
}
```

A repository is not a database connection. It uses one.

Mistake 5: Thinking Parent Reference Can Access All Child Methods

Wrong expectation:

```
Animal animal = new Dog();  
animal.fetch(); // assuming this works
```

It works only if `fetch()` exists in `Animal`.

The compiler checks the reference type first.

19. Placement-Ready Short Answers

What Is Method Overloading?

Method overloading is a compile-time polymorphism feature where multiple methods have the same name but different parameter lists in the same class.

What Is Method Overriding?

Method overriding is a runtime polymorphism feature where a subclass provides its own implementation of a method already defined in its parent class.

What Is Compile-Time Polymorphism?

Compile-time polymorphism means the compiler decides which method to call based on the method signature. It is commonly achieved through method overloading.

What Is Runtime Polymorphism?

Runtime polymorphism means the method call is resolved at runtime based on the actual object type. It is commonly achieved through method overriding.

What Is Association?

Association is a general relationship between two classes where one class is connected to, uses, or communicates with another class.

What Is Aggregation?

Aggregation is a weak has-a relationship where the child object can exist independently of the parent object.

What Is Composition?

Composition is a strong has-a relationship where the child object's lifecycle depends on the parent object.

Difference Between Aggregation And Composition?

In aggregation, the child can exist independently of the parent. In composition, the child is strongly owned by the parent and usually cannot exist meaningfully without it.

Difference Between Overloading And Overriding?

Overloading means same method name with different parameters and is resolved at compile time. Overriding means redefining a parent method in a child class and is resolved at runtime.

20. Practice Questions

Conceptual Questions

1. What is polymorphism?
2. What are the two types of polymorphism?
3. Why is overloading called compile-time polymorphism?
4. Why is overriding called runtime polymorphism?

5. Can return type alone overload a method?
6. Can constructors be overloaded?
7. Can constructors be overridden?
8. Can private methods be overridden?
9. Can static methods be overridden?
10. What is method hiding?
11. What is dynamic method dispatch?
12. What is covariant return type?
13. What is the difference between association and aggregation?
14. What is the difference between aggregation and composition?
15. Why is composition often preferred over inheritance?

Output-Based Questions

Question 1:

```
class Demo {  
    void show(int x) {  
        System.out.println("int");  
    }  
  
    void show(double x) {  
        System.out.println("double");  
    }  
  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.show(10);  
    }  
}
```

Answer:

```
int
```

Question 2:

```
class Parent {  
    void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    void show() {  
        System.out.println("Child");  
    }  
}
```

```
}

class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.show();
    }
}
```

Answer:

Child

Question 3:

```
class Animal {}
class Dog extends Animal {}

class Demo {
    void show(Animal a) {
        System.out.println("Animal");
    }

    void show(Dog d) {
        System.out.println("Dog");
    }

    public static void main(String[] args) {
        Animal a = new Dog();
        Demo demo = new Demo();
        demo.show(a);
    }
}
```

Answer:

Animal

Reason:

Overloading is resolved using the compile-time reference type.

Question 4:

```
class Parent {
    static void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    static void show() {
        System.out.println("Child");
    }
}

class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.show();
    }
}
```

Answer:

Parent

Reason:

Static methods are hidden, not overridden.

21. Final Revision Sheet

Polymorphism = many forms.

Overloading:

- Same method name
- Different parameters
- Same class usually
- Compile-time polymorphism
- Return type alone is not enough

Overriding:

- Same method signature
- Parent-child relationship
- Runtime polymorphism
- Object type decides method
- Static/private/final methods are not overridden

Compile-time polymorphism:

- Early binding

- Compiler decides
- Overloading

Runtime polymorphism:

- Late binding
- JVM decides
- Overriding

Association:

- General relationship
- A uses B
- Example: Teacher teaches Student

Aggregation:

- Weak has-a
- Child can live independently
- Example: Department has Professors

Composition:

- Strong has-a
- Child depends on parent
- Example: House has Rooms

Inheritance:

- Is-a
- Example: Dog is an Animal

Composition:

- Has-a
- Example: Car has an Engine

22. Best Interview Explanation Flow

If an interviewer asks about these topics, answer in this order:

1. Give a one-line definition.
2. Mention the type of relationship or polymorphism.
3. Give a simple example.
4. Mention one rule or exception.
5. Connect it to design if possible.

Example answer for overriding:

Method overriding means a child class provides its own implementation of a method already present in the parent class. It is an example of runtime polymorphism because the method call is resolved based on the actual object at runtime. For example, if Animal has sound() and Dog overrides sound(), then Animal a = new Dog(); a.sound(); calls Dog's sound method. Static methods are not overridden;

they are hidden. Overriding is useful in LLD because it lets us depend on parent types or interfaces while changing behavior through child classes.

That style is concise, accurate, and interview-friendly.